UML Clases - Seguridad RBAC/JWT (Semana 10)

Fecha: 2026-05-15

```mermaid
classDiagram
  class AppUser {
    +String id
    +String email
    +String fullName
    +Set~String~ rolesLegacy
    +Boolean enabled
  }

  class AppRole {
    +String id
    +String code
    +String name
    +Boolean system
  }

  class AppPermission {
    +String id
    +String code
    +String module
    +String name
  }

  class UserRoleAssignment {
    +String userId
    +String roleId
    +Instant assignedAt
  }

  class RolePermission {
    +String roleId
    +String permissionId
  }

  class UserVerification {
    +String userId
    +VerificationStatus status
    +Boolean emailVerified
    +Boolean phoneVerified
    +Boolean identityVerified
  }

  class VerificationEvent {
    +String id
    +String userId
    +String eventType
    +String payloadJson
    +Instant createdAt
```

```
  }

  class JwtAuthenticationFilter {
    +doFilterInternal()
  }

  class AuthorizationResolverService {
    +resolveForUser(userId)
  }

  class SecurityIntegrationConfig {
    +securityFilterChain()
  }

  class AccessAdminService {
    +listUsers()
    +listRoles()
    +listPermissions()
    +updateUserRoles(userId, roleCodes)
  }

  AppUser "1" --> "0..*" UserRoleAssignment
  AppRole "1" --> "0..*" UserRoleAssignment
  AppRole "1" --> "0..*" RolePermission
  AppPermission "1" --> "0..*" RolePermission
  AppUser "1" --> "0..1" UserVerification
  AppUser "1" --> "0..*" VerificationEvent

  JwtAuthenticationFilter --> AuthorizationResolverService
  SecurityIntegrationConfig --> JwtAuthenticationFilter
  AccessAdminService --> AppRole
  AccessAdminService --> AppPermission
  AccessAdminService --> UserRoleAssignment
```

Resumen

- El modelo separa autenticacion (JWT) de autorizacion (RBAC en BD).
- Los permisos no quedan hardcodeados en el frontend ni en el token.
- Queda preparado para evolucionar a trust score y auditoria avanzada.